

OOP : Assignment 1

1. What is a Programming Paradigm, and Why is it Important in Python?

→ A programming paradigm is a fundamental approach to writing and structuring code. Python supports multiple paradigms:

- Procedural (step-by-step execution with functions)
- Object-Oriented (OOP) (using classes and objects)
- Functional (using pure functions and immutability)

✦ Why is it important? Python's flexibility allows developers to choose the best approach based on project needs, making it widely used in various fields like web development, AI, and automation.

2. Explain the key difference between Object-Oriented Programming (OOP) and Procedural Programming.

Feature	OOP (Object-Oriented)	Procedural
Focus	Objects & data	Functions & procedures
Structure	Uses classes & objects	Uses functions & logic flow
Code Reusability	High (inheritance & polymorphism)	Lower (code repetition)
Encapsulation	Yes (data hiding)	No (global data access)
Example	Car class with attributes like color & model	Functions calculating sum, average, etc.

3. What are some advantages of using the OOP paradigm in Python?

- **Code Reusability** – Use inheritance to avoid redundancy.
Encapsulation – Protects data from unintended modification.
Modularity – Easier to organize and manage code.
Scalability – Efficient handling of complex programs.
✔ **Data Abstraction** – Hides complex logic, exposing only needed functionality.
-

4. How does Python implement data hiding in classes?

→ Python does not strictly enforce data hiding but allows it using private attributes (prefix `_` or `__`).

```
class Example:
    def __init__(self):
        self.__hidden = 10 # Private variable (cannot be accessed directly)

obj = Example()
print(obj.__hidden) # ❌ This will cause an AttributeError
```

→ Instead, access through **getter/setter methods**:

```
class Example:
    def __init__(self):
        self.__hidden = 10

    def get_value(self):
        return self.__hidden

obj = Example()
print(obj.get_value()) # ✅ Correct way to access private data
```

5. What is a class in Python, and how is it defined?

→ A class is a blueprint for creating objects, defined using the class keyword.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand # Instance Variable
        self.model = model

    def display_info(self):
        print(f"Car: {self.brand} {self.model}")

# Creating an object
car1 = Car("Toyota", "Corolla")
car1.display_info() # Output: Car: Toyota Corolla
Classes help in code organization, reusability, and data abstraction.
```

6. How is an object created from a class in Python?

An object is created by calling the class as if it were a function, using the syntax: `object_name = ClassName()`. This calls the `__init__()` method to initialize the object.

7. What role does the self keyword play in Python class methods?

`self` represents the instance of the class within its methods. It allows the instance to access its own attributes and methods, making it possible to work with specific data tied to that object.

8. What is the purpose of the init() method in Python classes?

The `__init__()` method is a special constructor method in Python. It is automatically called when an object is instantiated, and it is used to initialize the object's attributes.

9. How does Python achieve inheritance in OOP?

Python achieves inheritance by allowing a class (child class) to inherit attributes and methods from another class (parent class) using the syntax: `class ChildClass(ParentClass):`

10. What is polymorphism in Python, and how is it useful?

Polymorphism allows different classes to define methods with the same name, but each method can have different behaviors. It enables code flexibility and the ability to use objects of different types interchangeably.

11. Describe the concept of encapsulation in Python.

Encapsulation is the concept of bundling data and methods that operate on that data within a single unit (class). It also hides the internal state of the object, only exposing necessary details via public methods.

12. What is abstraction, and how does Python implement it?

Abstraction simplifies complex systems by exposing only the essential parts. Python implements abstraction through abstract classes and methods, typically using the `abc` module, where the implementation details are hidden from the user.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
```

```
def start(self):  
    pass
```

13. How does Python handle memory allocation for objects?

Python automatically handles memory management through a garbage collection mechanism, which keeps track of object references and frees memory when objects are no longer in use.

14. What is the purpose of a constructor in Python?

The constructor, typically the `__init__()` method, is used to initialize the attributes of a newly created object. It is called automatically when an object is instantiated.

15. How can you define a constructor in Python?

A constructor in Python is defined as the `__init__()` method within a class. It takes `self` as its first parameter and can take additional parameters to initialize the object.

```
class Car:  
    def __init__(self, model):  
        self.model = model
```

16. What happens when an object is deleted in Python?

When an object is deleted using `del`, the reference to the object is removed, and if there are no other references to the object, it is garbage collected, freeing its memory.

```
del obj
```

17. How can you explicitly delete an object in Python?

You can delete an object explicitly using the `del` keyword: `del object_name`. This removes the reference to the object and can trigger garbage collection if no other references exist.

18. What is the `del()` method used for in Python?

The `__del__()` method is a destructor that is called when an object is about to be destroyed. It is used for cleanup operations, such as closing files or releasing external resources.

19. What is the purpose of the super() function in Python?

The super() function is used to call a method from a parent class. It allows you to invoke methods from a superclass without explicitly naming it, supporting method resolution order (MRO) in inheritance.

20. How can you create a child class that inherits from a parent class in Python?

You can create a child class by specifying the parent class in parentheses when defining the child class:

```
class ChildClass(ParentClass):
```

This allows the child class to inherit methods and attributes from the parent.

21. What is the difference between a class attribute and an instance attribute in Python?

A class attribute is shared by all instances of a class, whereas an instance attribute is unique to each object. Class attributes are defined within the class, while instance attributes are typically defined within the __init__() method.

22. How can you define a private variable in a Python class?

A private variable in Python is defined by prefixing the variable name with double underscores: __private_var. This triggers name mangling to prevent direct access from outside the class.

23. What is the purpose of the @staticmethod decorator in Python?

The @staticmethod decorator is used to define a method that does not need access to the instance (self) or class (cls). It behaves like a regular function, but it belongs to the class's namespace.

24. How is a static method different from an instance method in Python?

An instance method requires an instance (self) to access or modify instance-specific attributes, whereas a static method does not require an instance and operates independently of the class state.

25. Explain the role of the @classmethod decorator in Python.

The @classmethod decorator defines a method that receives the class (cls) as its first argument, instead of the instance. It can access and modify class attributes, but not instance-specific attributes.

26. How can you access private members of a class in Python?

Private members (prefixed with `__`) are not directly accessible outside the class, but they can be accessed using name mangling: `object._ClassName__private_var`.

27. What is a friend function, and how is it implemented in Python?

In Python, there is no direct equivalent of the C++ "friend function." However, a function can be defined outside the class but still have access to its private members if explicitly passed the class instance as an argument.

28. What is a prototype in Python, and why is it important in OOP?

A prototype is a template for creating objects, ensuring consistent structure and behavior, aiding in object creation and reuse in OOP.

29. What are arrays of objects in Python, and how are they useful?

Arrays of objects are lists that store multiple instances of a class, useful for managing collections of objects efficiently.

30. How can you create an array of objects in Python?

You can create an array of objects by storing instances of a class in a list: `objects_array = [Class(), Class()]`.

31. What is the purpose of the `new()` method in Python?

The `__new__()` method creates a new instance of a class, allowing customization of object creation before initialization in `__init__()`.

32. How does Python differ from other languages in terms of memory allocation for objects?

Python uses automatic memory management with garbage collection, unlike some languages, which require manual memory allocation and deallocation.

33. How can you create multiple objects from a single class in Python?

You can create multiple objects by calling the class constructor multiple times: `object1 = Class(), object2 = Class()`.

34. What happens if a class in Python has no constructor defined?

If no constructor is defined, Python automatically provides a default `__init__()` method that does nothing but allows object instantiation.

35. How can you create default values for object properties in Python?

Default values can be set in the `__init__()` method by assigning values to instance attributes: `self.attribute = default_value`.

36. What is the difference between a shallow copy and a deep copy in Python classes?

- **Shallow Copy:** Copies object, but nested objects are referenced, not copied.
 - **Deep Copy:** Recursively copies object and all nested objects independently.
 - **Shallow Copy:** Changes to nested objects affect the original object.
 - **Deep Copy:** Nested object changes do not affect the original object.
-

37. How can you copy an object in Python?

- **Shallow Copy:** Use `copy.copy()` for copying without nested object duplication.
 - **Deep Copy:** Use `copy.deepcopy()` to copy all nested objects recursively.
 - **Manual Copy:** Create custom copy methods for specific object types.
 - **Immutable Objects:** Immutable types don't need copying as they can't change.
-

38. What is method overriding in Python, and how is it implemented?

- **Method Overriding:** Subclass redefines parent class method to modify behavior.
 - **Implementation:** Define a method with the same name in subclass.
 - **Purpose:** Change or extend inherited method behavior in subclasses.
 - **Example:** `def method_name(self):` in subclass overrides the parent's method.
-

39. How does Python allow multiple constructors in a class?

- **Class Methods:** Use `@classmethod` for alternate constructors in the class.
- **Default Arguments:** Provide default values in `__init__()` to simulate constructors.
- **Static Methods:** Use static methods for specific object creation logic.
- **Factory Methods:** Use custom class methods for different construction patterns.

40. How can you prevent a class from being inherited in Python?

- **Final Class:** Use final decorator (from typing module) to prevent inheritance.
 - **Raising Exceptions:** Raise exceptions in class methods to block subclassing.
 - **Private Methods:** Make methods private to discourage subclass modifications.
 - **Documentation:** Document the class to indicate it's not meant for inheritance.
-

